# 2nd Year 1st Semester
# University of Dhaka

## Department of Computer Science and Engineering

## CSE 2105
## Computer Architecture and Microprocessor
## Session: 2023-2024

**Assignment Name:**

# ALU-32 Bit (Arithmetic and Logic Unit)
# Project Report

**Submitted to**
**Khaleda Ali**
**EEE, DU**

**Submission Date: 10 - 01 - 2026**


## Submitted by:

## Group -10 (Khudro_Processor)

1. **Jahidul Islam [Roll 01]**
2. **Soumik Deb [Roll 11]**
3. **Mishkatul Ferdousi [Roll 45]**
4. **Tukabbir Hossain Sadi [Roll 48]**

# ALU-32 Bit (Arithmetic and Logic Unit) Project Report

## Introduction

This project implements an Arithmetic and Logic Unit (ALU) using ARM assembly language. The ALU can perform a variety of operations, including arithmetic, logical, shift/rotate, and other specialized functions. It processes two input operands, `A` and `B`, based on a given opcode and outputs the result in a designated `RESULT` register. The ALU also handles flags and comparison outputs, enhancing its functionality. This report provides a detailed explanation of the ALU's design, implementation, and supported operations.

## ALU Architecture

The ALU consists of several functional blocks:

1. **Input Operands**: Two operands `A` and `B` are provided as inputs.
2. **Opcode**: The operation to be performed is determined by the opcode, which specifies the type of arithmetic, logical, or shift operation.
3. **Result and Flags**: The output of the operation is stored in the `RESULT` register, and the processor's status flags are stored in the `FLAGS` register.
4. **Comparison Output**: For comparison operations, the result is stored in the `CMP_OUT` register, which indicates whether the comparison was greater than, equal to, or less than the other operand.

## Code Explanation

The ALU code is divided into multiple sections:

1. **Exports and Imports**:
   Registers `A`, `B`, `OPCODE`, `RESULT`, `FLAGS`, and `CMP_OUT` are defined as exports for use throughout the ALU. The `main` and `ALU_Step` functions are exported for execution.

```
EXPORT  A
EXPORT  B
EXPORT  OPCODE
EXPORT  RESULT
EXPORT  FLAGS
EXPORT  CMP_OUT

EXPORT  main
EXPORT  ALU_Step    ; function that executes ALU once
```

2. **Main Execution Loop**:
   The `main` loop continuously calls the `ALU_Step` function, which executes the ALU operation based on the given opcode. This allows for real-time inspection and debugging.

```
AREA MAIN_CODE, CODE, READONLY, ALIGN=2

main
loop_main
     BL       ALU_Step          ; run ALU once using A, B, OPCODE
     B        loop_main    ; keep looping so debugger can inspect
```

3. **Opcode Decoder:**
   The `ALU_Step` function decodes the opcode and jumps to the corresponding operation. For example, if the opcode is `0`, it performs addition (`ALU_ADD`), if it's `1`, it performs subtraction (`ALU_SUB`), and so on

```
; DECODER
CMP  R5, #0
BEQ  ALU_ADD
CMP  R5, #1
BEQ  ALU_SUB
...
```

4. **Arithmetic Operations:**
   Arithmetic operations include addition, subtraction, increment, decrement, addition with carry, and subtraction with borrow. Each operation is implemented with appropriate ARM instructions.

```
ALU_ADD
     ADDS    R9, R3, R4
     B       STORE_RESULT

ALU_SUB
     SUBS    R9, R3, R4
     B       STORE_RESULT
...
```

5. **Logical Operations:**
   Logical operations like AND, OR, XOR, NOT, NAND, and NOR are implemented using ARM instructions such as `AND, ORR, EOR,` etc.

```
ALU_AND
     AND     R9, R3, R4
     B       STORE_RESULT

ALU_OR
     ORR     R9, R3, R4
     B       STORE_RESULT
...
```

6. **Shift and Rotate Operations:**
   The ALU performs logical shifts, arithmetic shifts, and rotate operations (both left and right). The shift amount is derived from operand B and masked to 4 bits.

```
SHIFT_LSL
     AND     R4, R4, #0xF
     LSL     R9, R3, R4
     B       STORE_RESULT
...
```

7. **Special Operations:**

   - **Barrel Shift**: A specialized shift operation.
   - **Multiplication (Booth's algorithm placeholder)**: A simple multiplication implementation.
   - **Comparator**: Compares the two operands and stores the result (greater than, equal to, or less than) in `CMP_OUT`.
   - **Parity Checker**: Checks the parity of operand `A` and stores the result in `RESULT`.

```
BARREL_SHIFT
     AND     R4, R4, #0xF
     LSL     R9, R3, R4
     B       STORE_RESULT

BOOTH_MUL
     MUL     R9, R3, R4
     B       STORE_RESULT
...
```

## Flags and Result Storage

After performing the operation, the result is stored in the `RESULT` register. The status flags are updated and stored in the `FLAGS` register, which is read from the ARM processor's Application Program Status Register (APSR).

```
STORE_RESULT
    STR     R9, [R6]          ; Store result
    MRS     R10, APSR         ; Read flags
    STR     R10, [R7]         ; Store flags
```

## Opcode Table

Here is the opcode table for the ALU operations:

### 1. Arithmetic Operations

| Opcode | Operation Name | What It Does | Usage in Code |
|---|---|---|---|
| 0 | ADD | RESULT = A + B | ALU_ADD |
| 1 | SUB | RESULT = A - B | ALU_SUB |
| 2 | INC | RESULT = A + 1 | ALU_INC |
| 3 | DEC | RESULT = A - 1 | ALU_DEC |
| 4 | ADC (Add with carry) | RESULT = A + B + C_in (C_in from FLAGS bit 0) | ALU_ADC |
| 5 | SBB (Sub with borrow) | RESULT = A - B - Borrow (from FLAGS bit 0) | ALU_SBB |

## 2. Logical Operations

| Opcode | Operation Name | What It Does | Usage in Code |
|---|---|---|---|
| 6 | AND | RESULT = A AND B | ALU_AND |
| 7 | OR | RESULT = A OR B | ALU_OR |
| 8 | XOR | RESULT = A XOR B | ALU_XOR |
| 9 | NOT | RESULT = NOT A | ALU_NOT |
| 10 | NAND | RESULT = NOT (A AND B) | ALU_NAND |
| 11 | NOR | RESULT = NOT (A OR B) | ALU_NOR |

## 3. Shift / Rotate Operations

| Opcode | Operation Name | What It Does | Usage in Code |
|---|---|---|---|
| 12 | LSL (Logical left) | RESULT = A << (B & 0xF) | SHIFT_LSL |
| 13 | LSR (Logical right) | RESULT = A >> (B & 0xF) (zero-fill) | SHIFT_LSR |
| 14 | ASL (Arithmetic left) | RESULT = A << (B & 0xF) | SHIFT_ASL |
| 15 | ASR (Arithmetic right) | RESULT = A >>_arith (B & 0xF) (sign-ext) | SHIFT_ASR |
| 16 | ROL (Rotate left) | RESULT = (A << n) | (A >> (16 – n)), n = B & 0xF |
| 17 | ROR (Rotate right) | RESULT = A ROR (B & 0xF) | SHIFT_ROR |

## 4. Barrel Shifter

| Opcode | Operation Name | What It Does | Usage in Code |
|---|---|---|---|
| 18 | BARREL_SHIFT | RESULT = A << (B & 0xF) | BARREL_SHIFT |

## 5. Sequential Multiplication (Booth placeholder)

| Opcode | Operation Name | What It Does | Usage in Code |
|---|---|---|---|
| 19 | MUL / Booth | RESULT = A * B | BOOTH_MUL |

## 6. Comparator Unit

| Opcode | Operation Name | Output Location | Encoding in CMP_OUT | Usage in Code |
|---|---|---|---|---|
| 20 | COMPARE | CMP_OUT | 0001 (1) → A > B, 0010 (2) → A == B, 0100 (4) → A < B | COMPARATOR, CMP_GT, CMP_EQ, CMP_LT |

## 7. Parity Checker

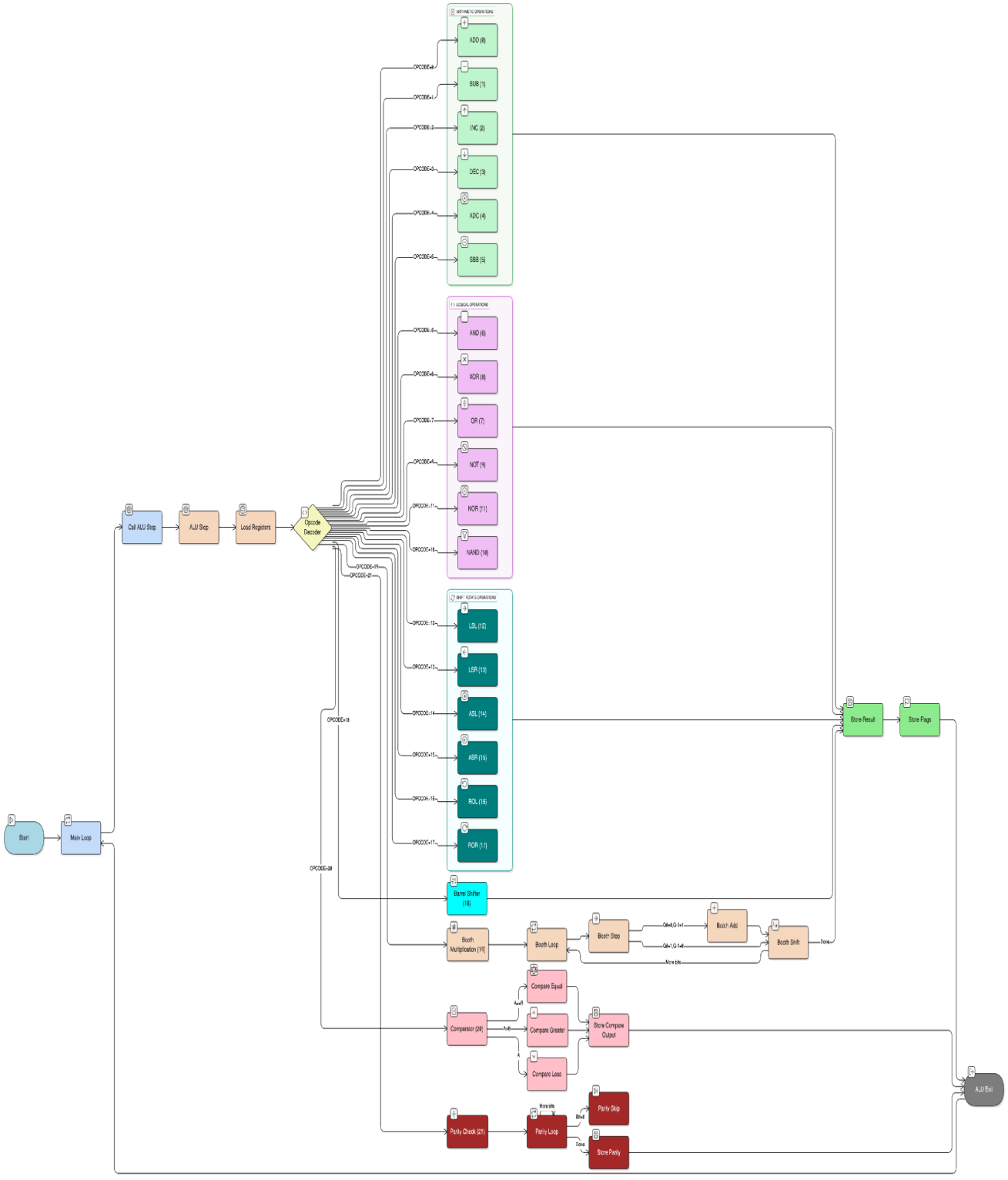
| Opcode | Operation Name | What It Checks | Where Stored | Usage in Code |
|---|---|---|---|---|
| 21 | PARITY | Parity of all bits of A | RESULT | PARITY_CHECK |

## Lesson Learned from This Project

This project has provided valuable lessons in both assembly language programming and hardware-level operations. Some of the key takeaways include:

1. **Understanding Low-Level Operations**:
   By implementing the ALU in ARM assembly, we gained a deeper understanding of how processors execute basic arithmetic and logic operations.
2. **Opcode Decoding and Control Flow**:
   Implementing opcode decoding taught us how processors use opcodes to control the execution of various operations based on input values, which is fundamental to processor design.
3. **Flag Management**:
   Managing the flags (such as carry and borrow) and understanding how they affect subsequent operations is a crucial part of processor behavior that ensures correct operation sequencing.
4. **Shift/Rotate and Bitwise Operations**:
   Implementing shift/rotate operations and bitwise logic deepened our understanding of how these operations are performed at the hardware level, often optimizing computations and memory usage.
5. **Practical Assembly Programming**:
   This project improved our practical skills in ARM assembly programming, including instruction set usage, memory management, and debugging.

**Flow Chart**




For Better Version :
https://github.com/Jahidul183019/CSE-2105-Computer-Architecture-Microprocessor-Assignment/blob/main/FlowChart.png

## Conclusion

This ALU project showcases the design and implementation of a fundamental unit that performs various arithmetic, logical, shift, and rotate operations using ARM assembly language. The modular structure of the ALU allows easy integration of additional operations, making it a versatile and essential component for more complex systems. The project emphasizes understanding the basics of processor-level operations, flags, and status registers.

## Demonstration

A video demonstration of the ALU project is available on YouTube:

https://youtu.be/6NoyZf5nCdE

### Here we appended the full project code :

```
    THUMB


    EXPORT  A
    EXPORT  B
    EXPORT  OPCODE
    EXPORT  RESULT
    EXPORT  FLAGS
    EXPORT  CMP_OUT

    EXPORT  main
    EXPORT  ALU_Step        ; function that executes ALU once


    ; MAIN SECTION
    AREA    MAIN_CODE, CODE, READONLY, ALIGN=2

main
loop_main
    BL      ALU_Step        ; run ALU once using A, B, OPCODE
    B       loop_main       ; keep looping so debugger can inspect


    ; ALU MODULE
    AREA    ALU_CODE, CODE, READONLY, ALIGN=2
```

```
        ; void ALU_Step(void)
        ;   Uses global A, B, OPCODE
        ;   Writes RESULT, FLAGS, CMP_OUT
ALU_Step
        PUSH    {R4-R11, LR}

        ; Load addresses
        LDR     R0, =A
        LDR     R1, =B
        LDR     R2, =OPCODE
        LDR     R6, =RESULT
        LDR     R7, =FLAGS
        LDR     R8, =CMP_OUT

        ; Load actual values
        LDR     R3, [R0]        ; A
        LDR     R4, [R1]        ; B
        LDR     R5, [R2]        ; OPCODE


        ; DECODER
        CMP     R5, #0
        BEQ     ALU_ADD
        CMP     R5, #1
        BEQ     ALU_SUB
        CMP     R5, #2
        BEQ     ALU_INC
        CMP     R5, #3
        BEQ     ALU_DEC
        CMP     R5, #4
        BEQ     ALU_ADC
        CMP     R5, #5
        BEQ     ALU_SBB

        CMP     R5, #6
        BEQ     ALU_AND
        CMP     R5, #7
        BEQ     ALU_OR
        CMP     R5, #8
        BEQ     ALU_XOR
        CMP     R5, #9
        BEQ     ALU_NOT
        CMP     R5, #10
        BEQ     ALU_NAND
        CMP     R5, #11
        BEQ     ALU_NOR

        CMP     R5, #12
```

```
        BEQ     SHIFT_LSL
        CMP     R5, #13
        BEQ     SHIFT_LSR
        CMP     R5, #14
        BEQ     SHIFT_ASL
        CMP     R5, #15
        BEQ     SHIFT_ASR
        CMP     R5, #16
        BEQ     SHIFT_ROL
        CMP     R5, #17
        BEQ     SHIFT_ROR

        CMP     R5, #18
        BEQ     BARREL_SHIFT

        CMP     R5, #19
        BEQ     BOOTH_MUL

        CMP     R5, #20
        BNE     NOT_OP20
        B       COMPARATOR
NOT_OP20

        CMP     R5, #21
        BNE     NOT_OP21
        B       PARITY_CHECK
NOT_OP21

        B       ALU_EXIT        ;


        ; 1. ARITHMETIC
ALU_ADD
        ADDS    R9, R3, R4
        B       STORE_RESULT

ALU_SUB
        SUBS    R9, R3, R4
        B       STORE_RESULT

ALU_INC
        ADDS    R9, R3, #1
        B       STORE_RESULT

ALU_DEC
        SUBS    R9, R3, #1
        B       STORE_RESULT
```

```
ALU_ADC
	LDR	R10, [R7]
	AND	R10, R10, #1
	ADDS	R9, R3, R4
	ADD	R9, R9, R10
	B	STORE_RESULT

ALU_SBB
	LDR	R10, [R7]
	AND	R10, R10, #1
	SUBS	R9, R3, R4
	SUB	R9, R9, R10
	B	STORE_RESULT


	; 2. LOGICAL
ALU_AND
	AND	R9, R3, R4
	B	STORE_RESULT

ALU_OR
	ORR	R9, R3, R4
	B	STORE_RESULT

ALU_XOR
	EOR	R9, R3, R4
	B	STORE_RESULT

ALU_NOT
	MVN	R9, R3
	B	STORE_RESULT

ALU_NAND
	AND	R9, R3, R4
	MVN	R9, R9
	B	STORE_RESULT

ALU_NOR
	ORR	R9, R3, R4
	MVN	R9, R9
	B	STORE_RESULT


	; 3. SHIFTS / ROTATES (B[3:0] = shift amount)
SHIFT_LSL
	AND	R4, R4, #0xF
	LSL	R9, R3, R4
	B	STORE_RESULT
```

```
SHIFT_LSR
    AND   R4, R4, #0xF
    LSR   R9, R3, R4
    B     STORE_RESULT

SHIFT_ASL
    AND   R4, R4, #0xF
    LSL   R9, R3, R4
    B     STORE_RESULT

SHIFT_ASR
    AND   R4, R4, #0xF
    ASR   R9, R3, R4
    B     STORE_RESULT

SHIFT_ROL
    AND   R4, R4, #0xF
    CMP   R4, #0
    BEQ   ROL_ZERO

    MOVS  R10, #16
    SUB   R10, R10, R4
    LSL   R9, R3, R4
    LSR   R11, R3, R10
    ORR   R9, R9, R11
    B     STORE_RESULT

ROL_ZERO
    MOV   R9, R3
    B     STORE_RESULT

SHIFT_ROR
    AND   R4, R4, #0xF
    CMP   R4, #0
    BEQ   ROR_ZERO

    ROR   R9, R3, R4
    B     STORE_RESULT

ROR_ZERO
    MOV   R9, R3
    B     STORE_RESULT


    ; 4. BARREL SHIFTER
BARREL_SHIFT
    AND   R4, R4, #0xF
```

```
        LSL     R9, R3, R4
        B       STORE_RESULT


        ; 5. BOOTH MULTIPLICATION (signed 32-bit)
BOOTH_MUL
        MOV     R9, #0
        MOV     R10, R4
        MOV     R12, #0
        MOV     R11, #32

BOOTH_LOOP
        ANDS    R0, R10, #1
        EOR     R1, R0, R12

        CMP     R1, #1
        BEQ     BOOTH_STEP
        B       BOOTH_SHIFT

BOOTH_STEP
        CMP     R0, #0
        BEQ     BOOTH_ADD       ; Q0=0,Q-1=1  -> ADD
        SUB     R9, R9, R3      ; Q0=1,Q-1=0  -> SUB
        B       BOOTH_SHIFT

BOOTH_ADD
        ADD     R9, R9, R3
        B       BOOTH_SHIFT

BOOTH_SHIFT
        AND     R2, R9, #1
        ASR     R9, R9, #1
        MOV     R1, R10, LSR #1
        ORR     R10, R1, R2, LSL #31
        MOV     R12, R0

        SUBS    R11, R11, #1
        BNE     BOOTH_LOOP

        MOV     R9, R10
        B       STORE_RESULT


        ; 6. COMPARATOR (CMP_OUT: 0001>, 0010==, 0100<)
COMPARATOR
        CMP     R3, R4
        BEQ     CMP_EQ
        BHI     CMP_GT
```

```
        BLO     CMP_LT

CMP_GT
        MOV     R9, #1
        STR     R9, [R8]
        B       ALU_EXIT

CMP_EQ
        MOV     R9, #2
        STR     R9, [R8]
        B       ALU_EXIT

CMP_LT
        MOV     R9, #4
        STR     R9, [R8]
        B       ALU_EXIT


        ; 7. PARITY CHECK (1 = even, 0 = odd) on A
PARITY_CHECK
        MOV     R10, R3
        MOV     R11, #1

PARITY_LOOP
        TST     R10, #1
        BEQ     PARITY_SKIP
        EOR     R11, R11, #1
PARITY_SKIP
        LSR     R10, R10, #1
        CMP     R10, #0
        BNE     PARITY_LOOP

        STR     R11, [R6]
        B       ALU_EXIT


        ; STORE RESULT + FLAGS
STORE_RESULT
        STR     R9, [R6]

        MRS     R10, APSR
        STR     R10, [R7]

        B       ALU_EXIT


        ; EXIT FROM FUNCTION
ALU_EXIT
```

```
    POP     {R4-R11, PC}


    ; DATA SECTION
    AREA    ALU_DATA, DATA, READWRITE, ALIGN=2

A         DCD     0x1234
B         DCD     0x00F3
OPCODE    DCD     0
RESULT    DCD     0
FLAGS     DCD     0
CMP_OUT   DCD     0

    END
```